



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Análisis y Desarrollo de Software
- **Código del Programa de Formación:** 228118
- **Nombre del Proyecto Formativo (si aplica):** Construcción de software orientado a servicios
- **Fase del Proyecto (si aplica):** Análisis
- **Actividad de Proyecto(si aplica):** Especificar el Modelo Conceptual del Software a Desarrollar
- **Competencia**
 - Evaluar requisitos de la solución de software de acuerdo con metodologías de análisis y estándares
- **Resultados de Aprendizaje:**
 - Modelar las funciones del software de acuerdo con el informe de requisitos
- **Duración de la Guía de Aprendizaje:** 60 Horas presenciales + 20 desescolarizadas

2. PRESENTACIÓN

El modelado del software es una etapa fundamental del proceso de desarrollo, ya que permite transformar los requerimientos del cliente en una representación estructurada del sistema. En esta guía, el aprendiz abordará el **análisis orientado a objetos**, el **modelado de clases mediante UML** y la **implementación de dichas clases en lenguajes como JavaScript y Python**, aplicando los principios de la **Programación Orientada a Objetos (POO)**.



3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

Una actividad de aprendizaje es una acción diseñada e intencionalizada que se realiza como un paso a paso para alcanzar un objetivo o una meta que pueda verse reflejado y aplicado, tanto en la parte conceptual (teoría), como en la parte procedimental (práctica), en esta guía las actividades están diseñadas como metas parciales y la suma de todas ellas llevan al logro de los resultado de los aprendizaje propuestos.

Es fundamental, que cada una de las actividades se realicen de forma comprensiva y crítica, de manera que en cada una de ellas se obtenga la información y los aprendizajes necesarios para responder a la construcción de evidencias de aprendizaje requeridas para la evaluación y aprobación de los resultados de aprendizaje de la competencia **“Evaluar requisitos de la solución de software de acuerdo con metodologías de análisis y estándares”**

¡Bienvenido/a y disfrute del aprendizaje!



- **Descripción de la(s) Actividad(es)**

3.1 Actividades de reflexión inicial:

Realizar inducción a la competencia “Evaluar requisitos de la solución de software de acuerdo con metodologías de análisis y estándares” **del programa Tecnólogo en Análisis y Desarrollo de Software**

Esta actividad se centra en que el aprendiz conozca el o los resultado(s) de aprendizaje a desarrollar, así como los conocimientos y criterios de evaluación asociados a cada uno de ellos.

Técnica Didáctica: Revisión de Material

Estrategia Didáctica: Aprendizaje colaborativo, gamificación

- El instructor de forma magistral dará a conocer la competencia “Evaluar requisitos de la solución de software de acuerdo con metodologías de análisis y estándares”.
- Se plantearán diferentes escenarios relacionados con la competencia para que el aprendiz participe activamente y al final conozca el alcance de los resultados de aprendizaje que componen la competencia.

Materiales para esta actividad: Computador, conectividad a internet, acceso a la herramienta LMS – TEAMS que tiene el SENA, diseño curricular del programa, guía de aprendizaje.

Tiempo sugerido para la actividad: 3 horas



3.2 Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje:

a. Identificar conocimientos previos relacionados con el diseño orientado a objetos

Esta actividad pretende identificar los conocimientos previos que tiene cada uno de los aprendices en la representación de la solución de software mediante objetos

Técnica Didáctica: Revisión de Material

Estrategia Didáctica: Aprendizaje colaborativo, gamificación

- Solución a las inquietudes presentadas en la actividad sobre aprendizajes previos

Responder a las siguientes inquietudes:

- ¿Qué es un actor?
- ¿Qué es una clase?
- ¿Conoce alguna herramienta para hacer diagramas UML?
- ¿Qué diagramas podemos realizar mediante UML?

Materiales para esta actividad: Computador, conectividad a internet, acceso a la herramienta LMS – TEAMS que tiene el SENA, guía de aprendizaje

Resultado esperado de esta actividad:

- Documento en formato pdf con la solución a las diferentes inquietudes presentadas en el documento de la actividad.

Tiempo sugerido para la actividad: 3 horas



3.3 Actividades de apropiación de Conocimiento

a. Modelar la solución de Software mediante diagramas UML - Clases

Técnica Didáctica: estudios de caso y proyectos

Estrategia Didáctica: Aprendizaje Autonomo y Colaborativo.

- El instructor con el apoyo del objeto virtual de aprendizaje “**Gestión de Requisitos**” hará una introducción al lenguaje de modelado UML.
- Revisar en su totalidad el objeto de aprendizaje
- Realizar la actividad didáctica propuesta por el objeto
- Modelar funciones mediante diagramas de clases de acuerdo con requisitos propuestos por el instructor
- Modelar las funciones del proyecto mediante diagramas de clases de acuerdo con el informe de requisitos elaborado en fases anteriores

Materiales para esta actividad: Computador, internet, acceso Microsoft Teams, objeto virtual de aprendizaje, informe de requisitos del proyecto

Objetos virtuales de Aprendizaje: Gestión de Requisitos

Resultado esperado de esta actividad:

- En un documento pdf entrega de pantallazo final de la solución de la actividad didáctica del objeto de aprendizaje
- Documento en formato pdf con los diferentes diagramas propuestos
- Diagramas de los diferentes ejercicios propuestos por el instructor para entregar como evidencia.
- Diagramas de clases de acuerdo con las funciones del proyecto

Tiempo sugerido para la actividad: 18 horas



b. Identificar y aplicar los conceptos del diseño y programación orientada a objetos

Técnica Didáctica: Estudio de caso o proyecto

Estrategia Didáctica: Aprendizaje autónomo y colaborativo.

- Revisar el objeto virtual de aprendizaje sobre “conceptos básicos de programación orientado a objetos”
- En grupos de dos personas exponer el concepto.
 - Herencia
 - Encapsulamiento
 - Polimorfismo
 - Clases Interfaces
 - Clases Abstractas
 - Sobrecarga
 - Cohesión
- Para cada uno de los ítems anteriores hacer un ejemplo que incluya diagrama de clases y código en lenguaje de programación (Javascript o Python)

Ambiente Requerido: Computador con conexión a internet, herramientas informáticas como office, entorno desarrollo visual studio code

Materiales para esta actividad: Documentos de apoyo y/o bibliografía.

Objetos Virtual de Aprendizaje:

Conceptos básicos de programación orientada a objetos

Resultado esperado de esta actividad:

- Presentación conceptos programación orientada a objetos Formato pdf.
- Código fuente de los ejercicios.

Tiempo sugerido para la actividad: 18 horas



c. Transformar modelos de clases mediante un lenguaje de programación

Técnica Didáctica: Estudio de caso o proyecto

Estrategia Didáctica: Aprendizaje autónomo y colaborativo.

- El instructor de forma magistral explicará como realizar la transformación de un diagrama de clases a un lenguaje de programación.
- Realizar los ejercicios propuestos en las sesiones de clase por parte del instructor
- Realizar los ejercicios propuestos para entregar como evidencia

Materiales para esta actividad: Computador, internet, acceso al LMS Microsoft Teams, uso de entorno de desarrollo visual studio code, lenguaje de programación Python o javascript.

Objeto virtual de Aprendizaje: “Aplicación paradigma POO”

Resultado esperado de esta actividad:

- Diagramas de clases
- Código fuente de las diferentes clases transformadas a partir del diagrama

Tiempo sugerido para la actividad: 24 horas

d. Evaluar Conocimientos de conceptos generales sobre paradigma orientado a objetos

Técnica Didáctica: Preguntas y respuestas

Estrategia Didáctica: Aprendizaje autónomo

- Presentación de la prueba publicada en el LMS.
- Al finalizar la prueba, la herramienta le informará el resultado

Materiales para esta actividad: Computador, internet, acceso a plataforma

Resultado esperado de esta actividad:

- Prueba presentada en el LMS

Tiempo sugerido para la actividad: 30 minutos



4. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN EN EL PROCESO FORMATIVO.

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
Análisis	Especificar el Modelo Conceptual del Software a Desarrollar	Modelar la solución de Software mediante diagramas UML - Clases	- Diagramas de clases propuestos por el instructor - Diagramas de clases de las funciones del proyecto.	Elabora diagramas de clase de acuerdo con los estándares actuales mediante Lenguaje Unificado de Modelado (UML). Representa el negocio en término de clases abstractas generando un modelo de dominio consistente	Listas de chequeo
		Identificar y aplicar los conceptos del diseño y programación orientada a objetos	- Documento con la presentación de la exposición conceptos de POO - Código fuente de los diferentes ejercicios sobre conceptos de POO		
		Transformar modelos de clases mediante un lenguaje de programación	- Diagramas de clases de los diferentes ejercicios propuestos - Código fuente de las diferentes clases transformadas a partir del diagrama		
		Evaluar Conocimientos de conceptos generales sobre paradigma orientado a objetos	- Presentar la evaluación de conocimientos sobre Diseño y POO		



5. GLOSARIO DE TÉRMINOS

Actividades: acción requerida para llevar a cabo un proceso.

Actor: se emplea para indicar el tipo de usuario del sistema que podrá ejecutar alguna función.

Asociación: relación que se da entre elementos de los diagramas, por ejemplo entre los actores y el caso de uso o entre las clases.

Atributos: características de una Clase, son datos específicos que interesa guardar de cada entidad.

Cardinalidad: indica la participación que tiene una entidad en la relación.

Caso de uso: indica una función que el sistema debe proveer.

Entidad: elemento del sistema de los cuales interesa almacenar información.

Extend: relación que se da entre casos de uso.

Relaciones: también se conoce como asociaciones, sirven para interconectar las entidades.

Transición: es un cambio de estado producido por un evento.

UML: Unified Modeling Language, es un lenguaje estandarizado que se utiliza para visualizar los elementos de un sistema de software , compuesto por diagramas que representan elementos estáticos y dinámicos del sistema

Abstracción: La característica específica de un objeto, aquella que lo distinguen de los demás tipos de objetos y que logran definir límites conceptuales respecto a quien está haciendo dicho abstracción del objeto.



Atributos: Características de una Clase, son datos específicos que interesa guardar de cada entidad. Aplican al objeto solo en el caso en que él sea visible en pantalla por el usuario; entonces sus atributos son el aspecto que refleja, tanto en color, tamaño, posición, si está o no habilitado.

Clase: Descripción de objeto. Unidad básica que agrupa una colección de objetos. Consta de una serie de métodos y datos que resumen las características de este objeto.

Encapsulación: Define el comportamiento de una clase u objeto que tiene dentro de él todo tipo de métodos y datos pero que solo es accesible mediante el paso de mensajes y los datos a través de los métodos del objeto/clase.

Herencia: Mecanismo para compartir automáticamente métodos y datos entre clases, subclases y objetos. Permite crear nuevas clases introduciendo las variaciones con respecto a su clase padre.

Métodos: Son aquellas funciones que permite efectuar el objeto y que nos rinden algún tipo de servicio durante el transcurso del programa. Determinan a su vez como va a responder el objeto cuando recibe un mensaje.

Objetos: Se trata de un ente abstracto usado en programación que permite separar los diferentes componentes de un programa, simplificando así su elaboración, depuración y posteriores mejoras. A los objetos se les otorga ciertas características en la vida real.



6. REFERENTES BIBLIOGRÁFICOS

- Diagramas de clases con UML: <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/diagramas-de-clases-con-uml/>
- Diagramas de casos de uso con UML: <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/diagrama-de-casos-de-uso/>
- Desarrollo orientado a objetos con UML: <https://www.uv.mx/personal/maymendez/files/2011/05/umltotal.pdf>
- Kimmel, Paul. (2008). *Manual de UML Guía de Aprendizaje*. Editorial McGraw-Hill Professional Publishing. Recuperado de <http://site.ebrary.com/bdigital.sena.edu.co/lib/senavirtualsp/detail.action?docID=10433806&p00=manual+uml>
- Programación orientada a objetos en DART: <https://andygeek.com/posts/Programacion-orientada-a-objetos-en-dart/>
- Programación orientada a objetos en Python: <https://kinsta.com/es/blog/programacion-orientada-objetos-python/>
- **Tutorial Python:** <https://www.w3schools.com/python/>
- **Introducción a la programación con Python:** <https://elibro-net.bdigital.sena.edu.co/es/lc/senavirtual/titulos/124259>

7. CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	03-02-2026

8. CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)					